

Which Rewrite Made My Model Slower?

Finding and Blaming Bad Graph Rewrites in TorchInductor — Midterm Progress Report
(S598APE)

Chidera Ibe

October 8, 2026

1 Summary

The proposal promised (a) a measurement harness that runs TorchInductor on real models with its graph rewrites turned on and off, one at a time and in combinations, (b) an attribution tool that takes a measured slowdown and names the rewrite, or pair of rewrites, that caused it, and (c) as a stretch goal, learned guards. This report covers proposal items 1–5: the corpus and rewrite switches (item 1), the measurement harness (item 2), the first batch of tests (item 3), the interim results (item 4), and a working first version of the attribution tool (item 5).

Everything described here exists as code in the `rewrite_blame` package, is driven from a command line, and is exercised by 100 automated tests that pass on both machines used. The main findings so far:

- TorchInductor 2.14.1 exposes **227 individually switchable pattern-matcher rules** plus 26 configuration flags and 16 opt-in rewrite passes, for a universe of 269 switches. Each pattern rule is switched without modifying PyTorch, by wrapping the rule’s `extra_check` hook, and the harness records how often each rule fired or was suppressed in every compile.
- The attribution tool, delta debugging over *signed* switch changes with a timing judge, recovers planted culprits in all tests, including a genuine **pair interaction** that exists in Inductor today: attention-fusion rules 11 and 12 match the same graph, so disabling either one alone changes nothing, and only disabling both changes the compiled program (Section 5).
- On the models measured so far, most individual rewrites do not change the generated program at all (Section 4); the ones that do are the pattern rules that actually fire, the master pass switches, freezing, and a handful of scheduler knobs. This is useful: the attribution search only has to consider the rules that fired, which is a short list per model.

One deviation from the proposal. No GPU was available (neither the course VM nor my laptop has one), so every measurement in this report uses Inductor’s **CPU C++/OpenMP backend** rather than Triton. The rewrite passes under study (joint-graph, post-grad, pre-grad pattern matching, decompositions, freezing) are the same on both backends; what differs is the kernel generator. The code-statistics module parses both the C++ and the Triton wrapper formats and is unit-tested against both, so a GPU run later is a change of device, not of tooling. Two very different CPUs (an Apple M4 Pro and an x86 Xeon with AVX-512 and oneDNN) stand in for the proposal’s “two GPUs” repeatability check.

2 What was built

2.1 Rewrite switches (proposal item 1b)

Inductor applies its rewrites in three places: *pre-grad* FX passes on the captured graph, *joint-graph* passes after AOTAutograd, and *post-grad* passes before lowering; most of them are instances of a generic `PatternMatcherPass`, each holding a list of `PatternEntry` rules keyed by the operator they match. Reading the source showed that the pass applies a rule only if `entry.extra_check(match)` returns true. The harness therefore replaces each entry’s `extra_check` with a small gate object that (1) refuses the match when the rule’s switch is off, (2) counts a *fire* when the rule’s own check accepts and the switch is on, and (3) counts a *suppression* when the rule would have fired but was disabled. No PyTorch source is modified.

Rules are registered lazily, so discovery first compiles a tiny warm-up graph that touches attention, convolution/batch-norm, split/cat, matmul and dtype casts, then walks the `fx_passes` modules for every pass object and gates every entry. Rule names come from the entry’s handler function or the replacement’s `pattern_name` (e.g. `joint/joint_graph.patterns/_sfdp_pattern_11_inference`, `post_grad/pass_patterns[1]/reciprocal_sqrt_to_rsqrt`). Three further kinds of switch cover rewrites that are not pattern entries: configuration flags (`epilogue_fusion`, `reorder_for_locality`, `freezing`, `cpp.weight_prepack`, ...), opt-in “optimus” passes enabled through Inductor’s fusion-option dictionaries (`split/cat` normalisation, `decompose_mm_pass`, batch fusions), and master switches for whole pass groups. A *state* is the set of switches that are on; `default` reproduces Inductor’s behaviour.

Two details mattered for correctness. First, Inductor’s on-disk FX-graph cache is keyed by its configuration but not by pattern state; a cache hit also skips every pass, which hides which rules fired. The harness therefore adds a no-op custom pass whose `uuid()` hashes the switch state into the cache key, and disables the FX-graph cache during measurements (the generated C++ kernels are still cached by source hash, so recompiles cost seconds). Second, calling Inductor’s lazy initialisers directly poisons their `functools.cache` keys and produces “duplicate pattern” errors on the next real compile; discovery must go through a real compile.

2.2 Measurement harness (item 2)

One measurement is one model under one state in a fresh subprocess, so gates, lazy state and allocator state never leak between configurations. The worker compiles the model with `torch.compile`, captures the generated wrapper code, and records:

- **generated-code statistics:** number of fused kernels (`cpp_fused_*` or `triton_*`), per-kernel input/output/in-place pointer counts, static load and store sites, vectorisation, kernel category (pointwise/reduction), extern and fallback calls (`extern_kernels.mm`, `aten._scaled_dot_product_flash_attention`), oneDNN fused convolutions), and the bytes of intermediate buffers allocated in the wrapper (the “extra tensor written to memory” the proposal worries about);
- **Inductor’s own metrics** (bytes accessed, IR nodes before fusion, loop reorderings) and the `counters["inductor"]` deltas;
- **which rules fired or were suppressed** and the post-grad op histogram, used to verify that a switch really changed the graph;
- **correctness** against eager execution (`allclose`, `rtol/atol` 10^{-3}) under every state, so a “fast” configuration is never a wrong one;
- **time:** fixed thread count, 10 warm-up calls, then 5×10 timed calls with per-call medians, MAD, and percentiles.

Results are stored in SQLite keyed by model, machine fingerprint, state hash and timing protocol, so attribution never measures the same configuration twice; re-running an attribution is served entirely from the store.

Noise threshold. The baseline state is measured in seven separate processes. The slowdown threshold is $\tau = \max(3 \cdot \text{MAD}(\text{medians}), 1\% \text{ of the centre})$; the report also records the largest difference seen between two *identical* runs, which is the false-positive scale the threshold must exceed. Table 3 lists the measured values.

2.3 Attribution tool (item 5, started)

Given a fast state F and a slow state S , the candidate causes are the signed changes that turn F into S (+id = turn on, -id = turn off); this covers both “a new rewrite hurt” and “a rewrite that used to help was lost”. The tool runs Zeller and Hildebrandt’s *ddmin* over that change set. The judge says *slow* when the median time of F with the candidate changes applied exceeds the fast reference by more than τ ; a candidate landing within $\pm 50\%$ of τ is measured once more and the two medians averaged. The result is 1-minimal: removing any single culprit makes the slowdown disappear. A pair check then measures each culprit alone and reports an *interaction* when none reproduces the slowdown by itself. The report names the culprits and prints the kernel-level difference between F and F +culprits: kernel count and names, extern calls, intermediate bytes, and which rules changed their firing. A second judge type uses a deterministic compile-time metric (kernel count, allocation bytes) and is what the tests on planted slowdowns use, so they are immune to timing noise.

2.4 Model corpus (item 1a)

Table 1 lists the 24 registered models: six hand-written graphs that each plant one rewrite family, six torchvision models, and twelve HuggingFace configurations including LLM prefill and single-token (decode) shapes. Models are built from fixed configs with random weights, so nothing is downloaded and timings do not depend on weights. Decode graphs are single-token forwards without a KV cache for now.

3 Testing (item 3)

Tests are split into *unit* tests (no compilation; `tests/test_u_*.py`) and *Inductor* tests that compile small graphs (`tests/test_i_*.py`); Table 2 counts them. Highlights:

- **ddmin on synthetic cost models:** single culprit, pair-only culprit (flagged as an interaction), two independent culprits (1-minimal result), three-way interaction, a masking rewrite that speeds up, a noisy judge (18/20 seeds correct at $\tau = 3\sigma$), 25 random cost models checked for “slow and 1-minimal”, memoisation, and the logarithmic number of judge calls (a culprit among 256 changes in ≤ 40 evaluations).
- **counters:** the code-statistics parser on hand-written C++ and Triton wrapper fixtures with known kernel counts, pointer counts, loads/stores, extern calls, allocation bytes including a symbolic shape, and the diff that ignores kernel numbering.
- **noise threshold:** MAD with floor, quiet-machine floor, re-measure band of the timing judge, serialisation.
- **planted rewrites** (compiled): for each toy graph, the expected rule fires by default, its switch suppresses it, the post-grad graph changes in the predicted way (e.g. `rsqrt` becomes `sqrt+reciprocal`; the dtype round trip survives without `pointless_convert`; SDPA

model	family	size	description
attention_block	toy	tiny	manual attention (B=4,T=128,C=512,H=8)
conv_bn_relu_stack	toy	tiny	3x conv3x3-bn-relu on 8x32x56x56
split_cat_mlp	toy	tiny	split->4 linears->cat->linear
norm_mlp	toy	tiny	layernorm/gelu MLP with pointless convert/view and 1/sqrt
decode_mlp	toy	tiny	M=1 gated MLP (LLM decode shape)
cat_slice_cat	toy	tiny	cat/slice/split_with_sizes/cat chains
resnet18	vision	small	torchvision resnet18, batch 8, 224x224
resnet50	vision	medium	torchvision resnet50, batch 4, 224x224
mobilenet_v3_small	vision	small	torchvision mobilenet_v3_small, batch 8, 224x224
efficientnet_b0	vision	medium	torchvision efficientnet_b0, batch 4, 224x224
convnext_tiny	vision	medium	torchvision convnext_tiny, batch 4, 224x224
vit_b_16	vision	large	torchvision vit_b_16, batch 4, 224x224
bert_base	hf	medium	BERT-base encoder, batch 8, seq 128, attn=default
bert_base_eagerattn	hf	medium	BERT-base encoder, batch 8, seq 128, attn=eager
distilbert	hf	small	DistilBERT encoder, batch 8, seq 128, attn=default
gpt2_prefill	hf	medium	GPT-2 small prefill, batch 4, seq 128, attn=default
gpt2_prefill_eagerattn	hf	medium	GPT-2 small prefill, batch 4, seq 128, attn=eager
gpt2_decode	hf	small	GPT-2 small single-token step (no KV cache), batch 4, seq 1, attn=default
t5_small_encoder	hf	small	T5-small encoder, batch 8, seq 128, attn=default
smollm_135m_prefill	hf	medium	SmolLM-135M (LLaMA arch) prefill, batch 1, seq 128, attn=default
smollm_135m_prefill_eagerattn	hf	medium	SmolLM-135M (LLaMA arch) prefill, batch 1, seq 128, attn=eager
smollm_135m_decode	hf	small	SmolLM-135M single-token step (no KV cache), batch 1, seq 1, attn=default
qwen2_0_5b_prefill	hf	large	Qwen2.5-0.5B prefill, batch 1, seq 128, attn=default
qwen2_0_5b_decode	hf	medium	Qwen2.5-0.5B single-token step (no KV cache), batch 1, seq 1, attn=default

Table 1: Model corpus (random-init from fixed configs).

disappears and two `bmm` calls plus a softmax kernel appear without the attention rules; `decompose_mm_pass` removes `mm` calls on the decode MLP; freezing folds batch-norm into the convolutions), and outputs match eager under both states.

- **pipeline:** subprocess isolation, store caching (second call makes no new measurement), failure reporting, the planted `sfdp {11,12}` interaction recovered by attribution with a metric judge in fewer compiles than leave-one-out, a single config culprit hidden among no-op changes, switch-effect classification, and determinism of the switch registry across processes.

test file	tests
test_i_discover.py	17
test_i_pipeline.py	9
test_i_planted.py	9
test_u_attribute.py	40
test_u_codegen_stats.py	6
test_u_judge_stats.py	8
test_u_store.py	3
test_u_switches.py	8
total	100

Table 2: Automated tests (unit tests run in well under a second; Inductor tests compile toy graphs).

4 Results so far (item 4)

4.1 Noise

model	machine	runs	centre (ms)	MAD (ms)	τ (ms)	τ (%)	max $ \Delta $ identical (ms)
attention_block	Chideras-MacBook-Pro-2	7	0.926	0.006	0.017	1.8	0.029
decode_mlp	Chideras-MacBook-Pro-2	7	0.194	0.004	0.011	5.5	0.046
norm_mlp	Chideras-MacBook-Pro-2	7	1.058	0.006	0.018	1.7	0.047
resnet18	Chideras-MacBook-Pro-2	7	26.118	0.192	0.576	2.2	0.451

Table 3: Baseline noise from repeated identical runs in separate processes (Mac: Apple M4 Pro, 8 threads; fa26: course VM, 4-core Xeon Silver 4216).

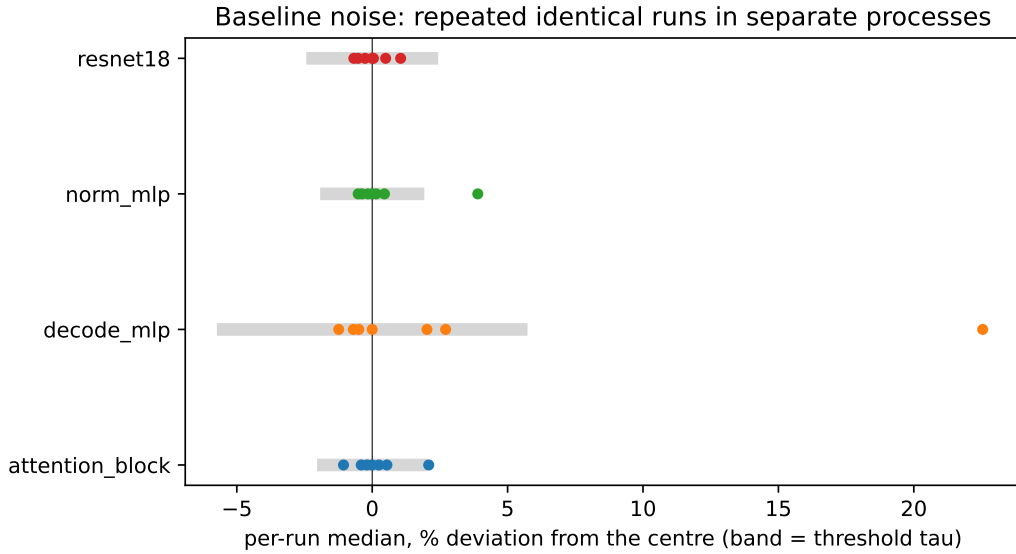


Figure 1: Per-run medians of identical baseline runs, as deviation from the centre; the grey band is τ .

4.2 Does flipping a switch change the program?

For each model the harness compiles the default state, then toggles every candidate switch (every rule that fired, every config flag, every opt-in option) and classifies the effect: *changes graph* (different kernels, extern calls, allocation or post-grad ops), *fires only* (a different rule fired but the program is the same), or *no effect*.

4.3 Leave-one-out timing sweeps

attention_block on the Mac (M4 Pro, 8 threads)

Baseline 0.926 ms, $\tau = 0.017$ ms (1.8%). 5 graph-changing switches were toggled one at a time: 2 made the model slower than τ , 0 made it faster, 3 stayed within noise. Slower: **master/pattern_matcher** (off, +33.1%); **master/post_grad_passes** (off, +3.2%).

model	kernels	rules fired	candidates (pat/cfg/opt)	changes graph	fires only	no effect	errors
attention_block	1	4	4/26/16	5	6	35	0
decode_mlp	1	0	0/26/16	4	0	38	0
norm_mlp	3	4	4/26/16	8	2	36	0
resnet18	14	1	1/26/16	4	3	36	0

Table 4: Switch verification: effect of toggling each candidate switch on the compiled program. A pattern rule that never fired cannot change anything when turned off, so only fired rules are candidates.

switch	toggled	median (ms)	Δ (%)	kernels	vs. τ
master/pattern_matcher	off	1.232	+33.1	4	slower
master/post_grad_passes	off	0.955	+3.2	2	slower
optimus/normalization_pass	on	0.916	-1.1	1	
sched/inplace_buffers	off	0.928	+0.3	1	
lowering/freezing	on	0.925	-0.0	1	

Table 5: Leave-one-out sweep, attention_block, Mac (M4 Pro, 8 threads).

decode_mlp on the Mac (M4 Pro, 8 threads)

Baseline 0.194 ms, $\tau = 0.011$ ms (5.5%). 4 graph-changing switches were toggled one at a time: 1 made the model slower than τ , 0 made it faster, 3 stayed within noise. Slower: `optimus/decompose_mm_pass` (on, +15.0%).

switch	toggled	median (ms)	Δ (%)	kernels	vs. τ
optimus/decompose_mm_pass	on	0.223	+15.0	1	slower
master/post_grad_passes	off	0.191	-1.6	1	
lowering/freezing	on	0.191	-1.4	1	
sched/inplace_buffers	off	0.191	-1.1	1	

Table 6: Leave-one-out sweep, decode_mlp, Mac (M4 Pro, 8 threads).

norm_mlp on the Mac (M4 Pro, 8 threads)

Baseline 1.058 ms, $\tau = 0.018$ ms (1.7%). 8 graph-changing switches were toggled one at a time: 2 made the model slower than τ , 0 made it faster, 6 stayed within noise. Slower: `sched/inplace_buffers` (off, +4.3%); `joint/constant_folding` (off, +2.6%).

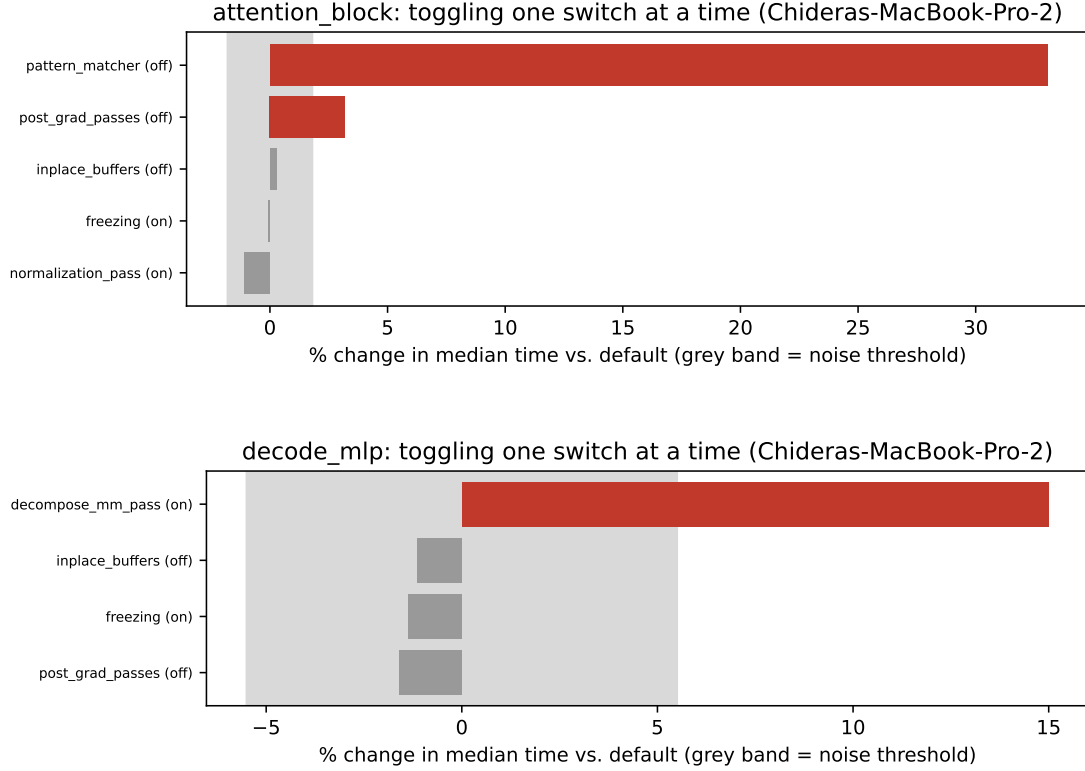
resnet18 on the Mac (M4 Pro, 8 threads)

Baseline 26.118 ms, $\tau = 0.576$ ms (2.2%). 4 graph-changing switches were toggled one at a time: 1 made the model slower than τ , 0 made it faster, 3 stayed within noise. Slower: `lowering/layout_optimization` (off, +11.1%).

5 Attribution case studies

attention_block on the Mac (M4 Pro, 8 threads): verdict *interaction*

The slow state differs from the fast state in 63 switches. Fast 0.919 ms vs. slow 1.250 ms (+36.1%); $\tau = 0.017$ ms. ddmin needed 16 judge evaluations (versus 63 for leave-one-out, 1953 for all pairs) and



returned the culprit set `-joint/joint_graph.patterns/_sfdp_pattern_11_inference`, `-joint/joint_graph.p`. Each culprit alone was judged *fast*: the slowdown needs all of them together. **Kernel-level explanation.** fused kernels `1 → 4`, extern calls `3 → 4`, intermediate bytes `4,198,400 → 11,591,680`. Kernels gained: `cpp_fused__softmax_abs_all_amax_div_eq_mul_ne_sub_view`, `cpp_fused_clone_split_transpose_v`, `cpp_fused_clone_transpose_view`. Extern calls lost: `torch.ops.aten._scaled_dot_product_flash_attention`. Kernels gained: `extern_kernels.bmm`, `extern_kernels.bmm`. Rules suppressed in the culprit state: `_sfdp_pattern_11_inference`, `_sfdp_pattern_12_inference`.

6 Status against the proposal and plan to the final report

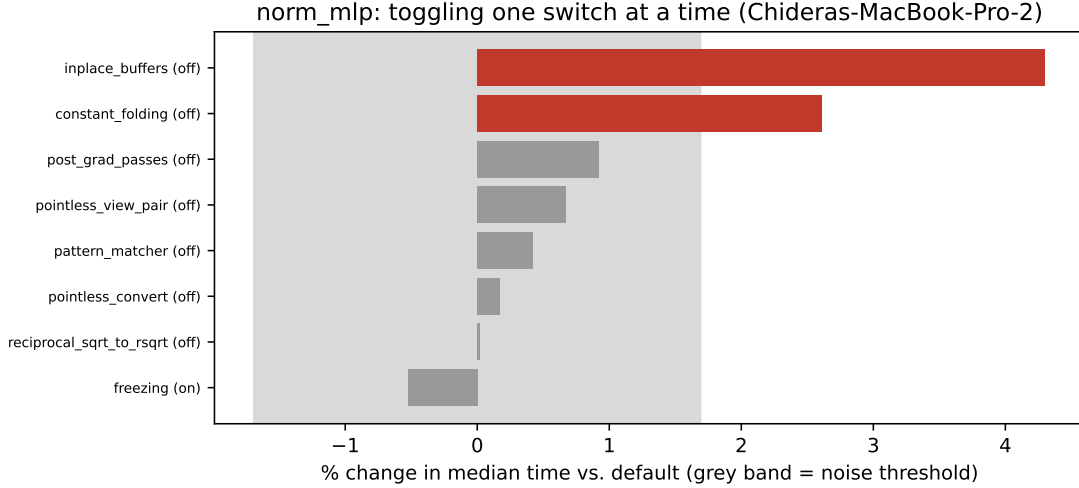
Risks and next steps. (1) Timing noise on the shared 4-core VM is the main threat to attribution; the mitigation is the measured τ , the re-measure band, and running the sweeps on both machines. (2) Most individual rewrites are no-ops on a given model, so slowdowns on real models will come from a few rules (attention fusion, freezing/oneDNN fusion, `decompose_mm`, split/cat passes); the next sweeps turn these opt-in families on in combination to look for pairs, as proposed. (3) Decode graphs should use a real KV cache. (4) If GPU access appears, the same commands run with `-device cuda` after a one-line change in the model builders.

References

- [1] O. Zinenko et al. The MLIR Transform Dialect. arXiv:2409.03864, 2024.
- [2] A. Vohra et al. Mind the Abstraction Gap: Bringing Equality Saturation to Real-World ML Compilers. OOPSLA 2025.

switch	toggled	median (ms)	Δ (%)	kernels	vs. τ
sched/inplace_buffers	off	1.103	+4.3	3	slower
joint/constant_folding	off	1.085	+2.6	3	slower
master/post_grad_passes	off	1.067	+0.9	3	
pointless_view_pair	off	1.065	+0.7	3	
lowering/freezing	on	1.052	-0.5	3	
master/pattern_matcher	off	1.062	+0.4	3	
pointless_convert	off	1.059	+0.2	3	
reciprocal_sqrt_to_rsqr	off	1.058	+0.0	3	

Table 7: Leave-one-out sweep, norm_mlp, Mac (M4 Pro, 8 threads).



- [3] Z. Jia et al. TASO. SOSP 2019.
- [4] PyTorch. `torch/_inductor/compiler_bisector.py`, 2025.
- [5] A. Zeller and R. Hildebrandt. Simplifying and Isolating Failure-Inducing Input. IEEE TSE, 2002.
- [6] J. Ansel et al. PyTorch 2. ASPLOS 2024.

switch	toggled	median (ms)	Δ (%)	kernels	vs. τ
lowering/layout_optimization	off	29.007	+11.1	12	slower
lowering/freezing	on	25.827	-1.1	11	
sched/inplace_buffers	off	26.239	+0.5	14	
master/post_grad_passes	off	26.099	-0.1	14	

Table 8: Leave-one-out sweep, resnet18, Mac (M4 Pro, 8 threads).

field	value
model	attention_block (Chideras-MacBook-Pro-2)
judge	timing (median_ms)
fast reference	0.926 ms, $\tau = 0.017$ ms (1.8%)
slow state	1.250 ms
candidate changes	63
judge evaluations	16
verdict	interaction
culprits	<code>_sfdp_pattern_11_inference</code> , <code>_sfdp_pattern_12_inference</code>
kernels fast $\rightarrow$ culprits	1 $\rightarrow$ 4
extern calls	3 $\rightarrow$ 4
intermediate bytes	4,198,400 $\rightarrow$ 11,591,680
median ms	0.919 $\rightarrow$ 1.244

Table 9: Attribution summary, attention_block.

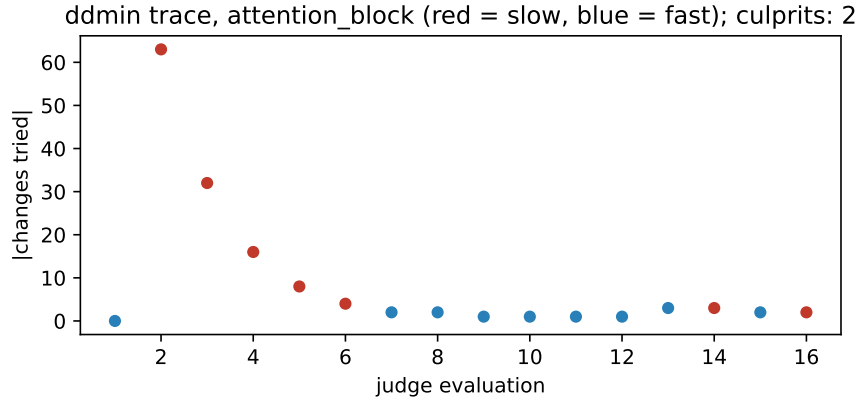


Figure 2: ddmin search trace for attention_block: each point is one judge evaluation.

proposal item	status
1. Corpus (10–15 models) and a switch per rewrite rule, verified to change the graph	Done. 24 models registered, 269 switches, verification tool classifies each switch’s effect per model.
2. Measurement harness: cached compiles, kernel and byte counters, timing with warm-up/trials, noise threshold	Done on the CPU backend; counters also parse Triton output (unit-tested).
3. Testing task 1: ≥ 30 unit tests with planted slowdowns, counter and threshold tests	Done, growing: 100 tests (Table 2).
4. Interim report: harness on 3–4 models with one clean attribution	Done: this report.
5. Attribution tool: ddmin over rule sets, timing judge, pair check, kernel-level diff	Working first version; needs more real-model case studies and a sensitivity study of τ .
6. Testing task 2: full corpus, two largest models, repeatability across days and machines	Started: sweeps run on both machines; the largest models (ViT-B/16, Qwen2.5-0.5B) are registered but not yet measured.
7. Stretch: learned guards	Not started.
8. Final report, README, cleanup	README exists.